Thomas Nguyen
11888002
Professor Zhe Dang
Due 4/13/26

1

Decide if there is an ω-path where ◇(yellow ∨ □blue) holds

The formula says: eventually, from some point on, every node is yellow or every node is blue.

The color patterns that satisfy this are: all yellow, OR all blue after some point.

Algorithm:

1. Run SCC on G
2. For each looping SCC, check if all nodes in it are yellow, or all are blue
3. If any such SCC is reachable from the initial node, return YES
4. Otherwise return NO

This works because an ω-path eventually stays inside one SCC forever. So we just need a reachable looping SCC where all nodes satisfy the condition.

Decide if there is an ω-path where □◇(yellow ∨ □blue) holds

The formula says: infinitely often, there is some point where from then on every node is yellow or every node is blue.

Algorithm:

1. Run SCC on G
2. Drop all blue nodes from G to get G'
3. Run SCC on G'
4. Check if any looping SCC in G' that contains a yellow node is reachable from the initial node in G
5. If yes, return YES. Otherwise return NO

This works because we need the path to keep returning to a region where the condition holds, which means revisiting a looping SCC with yellow nodes infinitely often.

---

2. Decide if there is an ω-path passing red infinitely many times and blue only finitely many times

The path must eventually stop seeing blue nodes but keep seeing red nodes forever.

Algorithm:

1. Run SCC on G
2. For each looping SCC that contains at least one red node and zero blue nodes, check if it is reachable from the initial node
3. If any such SCC exists and is reachable, return YES
4. Otherwise return NO

This works because after finitely many blue nodes, the path must settle into a looping SCC. That SCC needs to have red nodes (to see red infinitely often) and no blue nodes (to stop seeing blue).

---

3. Decide if there is a good ω-path where infinitely many prefixes have `#red` = `#blue`

A good ω-path must return to a state where `#red` = `#blue` infinitely many times.

Algorithm:

1. Define L = { w | `#red` = `#blue` }. L is context-free.
2. Build a PDA for L
3. Take the product of G and the PDA
4. Check if there is a node v reachable from the initial node such that:
   - The initial node can reach v through a walk with `#red` = `#blue`
   - v can reach back to itself through a walk with `#red` = `#blue`
5. If such a v exists, return YES. Otherwise return NO

This works because we need infinitely many prefixes where the count balances out, meaning the path must keep cycling back through states where `#red` = `#blue`.

---

4. Decide if there is a bad ω-path where infinitely many prefixes have `#red` as a multiple of 5

A bad ω-path must return to a state where `#red` mod 5 = 0 infinitely many times.

Algorithm:

1. Define L = { w | `#red` mod 5 = 0 }. L is regular.
2. Build a DFA M with 5 states tracking `#red` mod 5
3. Take the product of G and M
4. Check if there is a node v reachable from the initial node such that:

- The initial node can reach v with `#red` mod 5 = 0
- v can reach back to itself with `#red` mod 5 = 0

5. If such a v exists, return YES. Otherwise return NO

This works because we need the path to cycle back to a balanced state infinitely often. Since the language is regular, a simple DFA with 5 states is enough to track it.